

LATTICRA

The Latticra System Substrate

An Effect at Modern Security

A computer scientist's guide to evidence-bound, effect-aware project security

Evidence-first architecture for local integrity, policy boundaries, reproducible reports, and future runtime-handoff research.

Working Draft 0.12 - 2026-05-26

Front Matter

This book overhauls the original Latticra Seal handbook into a project-level explanation of Latticra as a system substrate. It is written for computer scientists: readers who care about invariants, authority boundaries, reproducible evidence, policy semantics, and the difference between measuring a system and enforcing a system.

The project should be understood as early and evidence-bound. The text deliberately avoids calling Latticra a production security product, a host-protection system, a sandbox, a kernel enforcement layer, a ransomware prevention system, or a malware prevention system. The value proposition presented here is narrower and more interesting: Latticra is a substrate for making security-relevant effects visible, typed, reviewable, and eventually governable.

Source posture: this edition draws from the Seal handbook plus current Seal signed-request, request-freshness, verification-receipt, crypto-verify-backend, Ed25519 verify-only, verified-receipt-promotion, verified-capability-gate, verified-effect-decision, runtime-handoff-evaluation, runtime-handoff-report, report-envelope, signature-request, capability-gate, and effect-decision records; Latticra Console Stage-0 host-adapter, receipt-request, receipt-payload, signature-request-binding, OS-base, and VM-evidence contract surfaces; Runtime Boundary Lat pipeline first-clause evidence propagation; Panel/installer receipts; Fedora and macOS validation material; Lat/LIR, Nucleus, Nadia, runtime-boundary, kernel-lifecycle, and defensive threat-model notes; and the one-pass field engine research artifact.

Living handbook cadence

This edition is a working artifact. As repository notes, status records, validation lanes, command contracts, and capability boundaries change, the durable changes should be folded back into this handbook instead of living only in scattered logs.

The update rule is evidence-bound: record what is implemented, tested, measured, planned, or explicitly out of scope, and keep future runtime, enforcement, host-protection, packaging, and production claims behind reproducible evidence.

Contents

Section	Title
Part I	Orientation
1	Why Latticra Exists
2	The Substrate Model
3	Current Evidence and Claim Ledger
Part II	Project Architecture
4	Latticra Components
5	Latticra Seal
6	Panel, Installer, and Receipts
7	Lat, LIR, and Contract Surfaces
8	Field Engine and Research Artifacts
Part III	Security Model
9	Authority Vectors and Effect Classes
10	Policy as a Fail-Closed Compiler
11	Reports, Receipts, and Witness Objects
12	Threats, Non-Goals, and Honest Boundaries
Part IV	Implementation Workbook
13	Command Contracts
14	Schemas and Formats
15	Test Lanes and Continuous Evidence
16	Roadmap and Research Agenda

Part I - Orientation

Latticra begins as a way to make system effects legible before it tries to make them enforceable.

Chapter 1. Why Latticra Exists

Modern systems fail as much from invisible authority and ambiguous claims as they do from isolated bugs. Latticra treats security posture as something that must be measured, constrained, documented, and reproduced before it is trusted.

Most security projects begin by promising protection. Latticra should begin with a more careful claim: it can help a developer and operator know what the project says it did, what evidence it can produce, and which authorities it explicitly did not use. This is not a weaker story. It is a computer-science story. It turns vague trust into state, transition, policy, and evidence.

The original Seal handbook presents Seal as the verification, reporting, and policy-boundary layer inside the ecosystem. This book expands that layer into a whole-project architecture: Latticra is a substrate whose primary job is to convert local project state into inspectable evidence and to keep unsupported authority claims from crossing a boundary unnoticed.

The phrase "an effect at modern security" is used here as a technical metaphor. In programming-language terms, an effect is something a computation does outside pure value production. Reading files, writing receipts, mutating host state, requesting network authority, installing wrappers, modifying kernel surfaces, and claiming runtime enforcement are effects. Latticra is useful when it makes those effects explicit and typed.

The project therefore addresses a recurring security engineering problem: a tool may be locally helpful, but nobody can quickly tell which authority it had, which state it observed, which files it changed, whether its report is reproducible, and whether the words around it outrun the tests. Latticra answers by making the default path evidence-first and claim-limited.

1.1 The central thesis

Latticra is an evidence-bound system substrate for describing, measuring, and presenting local project state under explicit authority constraints. It is not currently a production host defense product. Its importance is the discipline it introduces: before a project can enforce a security boundary, it must be able to name the boundary, test the boundary, report the boundary, and deny claims that exceed it.

- Evidence before enforcement: reports and receipts must precede security claims.
- Authority before mutation: root, network, runtime, kernel, systemd, and SELinux authority must be explicit.
- Policy before pass: a PASS means declared local expectations were met, not that the host is secure.
- Documentation before marketing: claims are bounded by implemented, tested, and reproducible behavior.

1.2 The reader model

This book assumes the reader is comfortable with operating systems, local package layouts, CI lanes, hashing, manifests, policy checks, and basic formal modeling. The goal is not to teach security from first principles. The goal is to explain the shape of Latticra as an artifact that computer scientists can inspect, criticize, extend, and test.

1.3 The smallest useful mental model

Figure 1.1 - Minimal Latticra model.

```
Project tree P
-> declared scope M
-> measured state H
-> policy decision D
-> report or receipt R
-> claim set C

Latticra is useful when: C <= evidence(R) <= test(D) <= implemented effects(P)
```

The model says that public claims must be no stronger than evidence. Evidence must be no stronger than tests. Tests must be no stronger than implemented effects. This creates a ladder that prevents the project from jumping directly from "we computed a hash" to "we secure the host."

Chapter 2. The Substrate Model

A substrate is a layer that other parts of a system can stand on. In Latticra, the substrate is not an operating system replacement; it is a vocabulary and evidence pipeline for system state, authority, and future security decisions.

The substrate idea is intentionally narrower than a platform claim. A platform normally promises user-facing capabilities. A substrate promises stable underlying relations: what can be observed, what can be recorded, what can be denied, and what future layers may consume. The Latticra substrate is a set of local mechanisms and design rules that encourage every meaningful effect to leave behind an intelligible witness.

2.1 Substrate objects

Object	Role in the substrate	Current posture
Manifest	Declares what the tool may inspect and what expectations apply.	Core Seal concept; schema should be hardened.
Lock/baseline	Captures hash material or expected state against which change is measured.	Present as hash lock material; stronger signed evidence is future work.
Policy	Accepts or denies state and authority requests.	Conservative; should fail closed.
Report	Human-readable and eventually machine-readable description of checks.	Report-only lane is central.
Receipt	Operator-facing record of installation or action.	Used by local installer evidence.
Panel	Human interface for plans, evidence, and guarded local actions.	Bridge/control surface, not a root authority.
Runtime handoff	Future metadata path for enforcement decisions.	Planned/research; not enforcement today.

These objects are not just files. They are an attempt to structure trust. A manifest limits scope. A lock makes drift visible. A policy denies unsupported states. A report tells the operator what happened. A receipt records what was installed or would have been installed. The Panel makes these relations visible instead of hiding them behind a button.

2.2 Security as an effect system

For a computer scientist, the natural abstraction is an effect system. A computation that only reads files has a different type from one that writes to a user prefix. A tool that writes receipts has a different type from one that modifies systemd. A UI that previews an install plan has a different type from one that uses root to mutate the host. Latticra can be described as an attempt to classify those effects and reject mismatches.

Effect class	Examples	Allowed current interpretation
ReadLocal	Read manifest, inspect project tree, read local config.	Allowed under declared scope.
MeasureLocal	Hash files, compare lock baseline, count changes.	Allowed when report-only and read-only.
WriteEvidence	Write report, plan, receipt, component marker.	Allowed when documented and local.
UserPrefixMutation	Create files under a guarded user-local prefix.	Allowed only by guarded installer profile.
NetworkOperation	Download, publish, call remote APIs.	Not a current authority lane.
RootMutation	Install as root, modify /usr, services, kernel, SELinux.	Not a current authority lane.
RuntimeEnforcement	Sandbox, block process, enforce isolation.	Future research; not currently claimed.

2.3 Why this matters

Security systems often degrade when documentation, implementation, and authority drift apart. Developers add flags. Installers add convenience. UI buttons hide side effects. CI proves a narrow property while README text promises a broad one. Latticra can become valuable because it treats that drift as the primary enemy. The substrate exists to keep claims, tests, and effects synchronized.

Chapter 3. Current Evidence and Claim Ledger

The strongest part of the project is not a large production guarantee. It is the explicit ledger of what exists, what is planned, and what must not be claimed yet.

The Seal handbook states a current readiness posture: local report generation, manifest/hash baseline, policy regression, and Panel bridge planning are positive claims. Runtime enforcement, network operation, root installation, kernel enforcement, and production readiness are zero claims. That ledger should not be a footnote. It should be the core of the system.

Capability or claim	Status in this edition	How to talk about it
Local report generation	Current evidence-bound capability	Seal can generate local evidence reports.
Manifest/hash baseline	Current evidence-bound capability	Seal can compare local evidence against declared expectations.
Policy regression lane	Current evidence-bound capability	Denied states are part of the test story.
Panel bridge	Planning plus installer/control-surface evidence	Panel should surface reports and authority state safely.
Guarded local-prefix installer	Evidence from local installer work	Installer can operate as user-local, guarded, and receipt-generating.
Runtime enforcement	Not current	Future handoff metadata only until enforcement exists and is tested.
Network authority	Not current	Do not claim network authority.
Root authority	Not current	Do not claim root installation.
Kernel/systemd/SELinux integration	Not current	Research or future integration only.
Production host security	Not current	Do not claim host protection, malware prevention, or ransomware prevention.
Nucleus task boundary	Report-only task records with no-effect alignment and execution/effect/runtime labels.	Say it classifies and reports task-shaped requests; do not say it executes tasks.
Nadia offline AI	Stage-0 through Stage-27 contract metadata; no model runtime, tokenization, context-window assembly, inference, dialogue, or tools.	Say it is an offline-AI contract ladder; do not say it is an active AI assistant.
macOS app bundle lane	No-effect user-local app-bundle writer dry-run, commit gate, reset/uninstall dry-run contract, and verification-transcript contract.	Say macOS install evidence is being specified; do not say a macOS installer exists.
Fedora RPM lifecycle lane	Controlled installroot/disposable-VM RPM evidence and package validation records.	Say bounded RPM lifecycle evidence exists; do not say Fedora approval or daily-driver readiness.
Production installer readiness	Contract-only readiness gate with required future artifacts, transcripts, signatures, rollback, and multi-VM evidence.	Say readiness is gated and currently false; do not say Latticra is production installable.
Seal signed request metadata	Report-only metadata; signing and verification unsupported.	Say Seal records signed-request vocabulary; do not say it signs, verifies, enforces, or trusts requests.
Seal verification receipt and crypto backend metadata	Baseline receipt/backend metadata remains unverified or unsupported until a later verify-only result is introduced.	Say the predecessor boundary is specified; do not treat backend metadata alone as cryptographic verification.

Seal capability and effect gates	Denied-by-default metadata with <code>gate_allowed=0</code> , <code>effect_allowed=0</code> , and <code>runtime_authority_granted=0</code> .	Say gates and decisions preserve non-effects; do not say they enforce capabilities or perform host effects.
Seal Ed25519 verify-only result	Local provider-backed verification of caller-supplied message, public key, and signature bytes; authority-neutral.	Say local Ed25519 verify-only evidence exists; do not claim signing, trust stores, revocation, production crypto, or authority.
Seal verified receipt promotion	Successful verify-only results can be promoted into verified receipt metadata with authority still zero.	Say evidence promotion; do not say permission promotion, capability authorization, effects, or runtime handoff.
Seal verified capability gate	Verified receipt metadata can pass a narrow report/evaluate metadata gate with <code>gate_allowed=1</code> and no effects.	Say metadata-only gate evaluation; do not say capability enforcement, effect execution, or runtime permission.
Seal verified effect decision	Allowed verified gate metadata can classify report/evaluate requests with <code>effect_allowed=1</code> while no effect is performed.	Say effect classification metadata; do not say effect execution, host effects, capability enforcement, or runtime permission.
Seal runtime handoff evaluation	Allowed metadata-only effect decisions can become handoff_eligible=1 with no handoff performed.	Say handoff eligibility classification; do not say runtime handoff execution or runtime authority.
Seal runtime handoff report	Eligible handoff metadata can become <code>report_ready=1</code> with no runtime handoff or effect execution.	Say report-readiness metadata; do not say runtime bridge, enforcement, or host/network behavior.
Seal report envelope	Ready runtime-handoff report metadata can become <code>envelope_ready=1</code> while unsigned and object-unsealed.	Say envelope classification metadata; do not say signing, object sealing, runtime handoff, or authority.
Seal signature request	Ready report-envelope metadata can request Ed25519-development signing as metadata with <code>signature_request_ready=1</code> and no signing.	Say signature-request classification; do not say Ed25519 signing, verification, private-key handling, or trust establishment.
LC host-adapter contract	Metadata-only contract; <code>host_adapter_present=0</code> , <code>host_process/file/network/runtime/boot</code> authority denied.	Say LC exposes the adapter boundary for review; do not say a host adapter exists, probes the host, or embeds into the host.
LC receipt and signature bridge	Metadata-only receipt request, payload schema, artifact draft, and signature-request binding; <code>payload_artifact_present=0</code> , <code>seal_signature_request_present=0</code> , <code>receipt_write_allowed=0</code> .	Say LC reserves the Seal receipt path; do not say it materializes payloads, writes receipts, signs, verifies, or hands off to a signer.
LC OS-base and VM evidence contracts	Metadata-only planning/evidence contracts; <code>os_base_enabled=0</code> , <code>vm_evidence_capture_enabled=0</code> , <code>boot_authority=0</code> , <code>production_os_claim=0</code> .	Say LC records OS/VM promotion gates; do not say it boots, launches VMs, touches hypervisors, writes disk images, or replaces an OS.
Runtime Boundary Lat clause evidence	Record-level propagation of the first lowered Lat clause node index, role, effect, name, operator, and value from pipeline evidence; no operator evaluation or runtime authority.	Say Runtime Boundary preserves clause provenance for review; do not say it evaluates Lat clauses, executes Lat/LIR, enforces policy, permits effects, or grants authority.

3.1 Claim discipline

A useful project book must not launder aspiration into implementation. For Latticra, aspiration is important: future cryptographic receipts, runtime-boundary metadata, and Linux integration research are all plausible directions. But they must remain future directions until a reader can reproduce the behavior.

The claim ledger should be kept in the repository as a first-class artifact. Every README, installer panel, report header, and public project page should be checked against it. When a capability moves from zero to one, the transition should require implementation, tests, documentation, and a short rationale.

3.2 The invariant behind the ledger

Algorithm 3.1 - Claim gate invariant.

For each claim c:
require evidence, tests, and implementation support
otherwise deny c or mark future

Part II - Project Architecture

The project is a collection of lanes: Seal, CLI contracts, Panel, installer, validation, and research artifacts. Their shared job is to preserve authority boundaries.

Chapter 4. Latticra Components

Latticra is best explained as a system of cooperating components rather than one monolithic security product.

The available project evidence exposes several component names and workflows: Latticra Seal, Latticra Console, Latticra CLI wrappers, Lat tooling, LIR contracts, the Latticra Panel, a guarded local-prefix installer, docs/examples, developer CLI helpers, Fedora/RPM validation lanes, and demonstration artifacts such as the one-pass field engine. This book treats those as separate lanes in a single substrate.

Component lane	Primary interface	Substrate responsibility
Seal	latticra-seal, reports, manifests, locks	Describe, measure, compare, deny unsupported claims, emit local evidence.
CLI / wrappers	latticra, lat, latticra-seal, latticra-panel, installer commands	Provide stable entry points and command contracts.
Lat tooling	lat command and related installed components	Project tooling lane with grammar, semantic, model, lowering, pipeline, diagnostic, and clause-provenance evidence surfaces.
LIR contracts	Installed contract docs and validation lanes	Intermediate representation and lowering evidence surface; preserves clause metadata without execution.
Panel	Graphical local installer/control surface	Expose plan, evidence, authority, and guarded actions to the operator.
Installer	dry-run, local-example, verify-local, uninstall-local	Build or install user-local payloads and receipts with explicit authority limits.
Validation	make targets, CI, Fedora VM/RPM lanes	Prove bounded claims and record non-claims.
Research artifacts	one-pass field engine and visual demos	Demonstrate concepts without claiming security boundaries.
LC	latticra-lc, command registry, profiles, host/receipt/OS contract reports	Operator command surface with no-effect metadata, host-adapter and receipt-request contracts, OS/VM evidence planning, and future gates.
Nucleus	nucleus task records and report-only runners	Coordinate task-boundary metadata without effect-performing execution.
Nadia	latticra-nadia, Panel component, Console metadata	Offline-AI contract ladder with denied model, tokenizer, prompt, runtime, and tool authority.
Platform install lanes	macOS contracts, Fedora RPM lanes, production readiness gates	Convert host-facing install claims into dry-run reports, bounded transcripts, and explicit non-readiness evidence.

4.1 The architecture as dataflow

Figure 4.1 - Latticra lanes as a conservative dataflow.

```

Repository and local prefix
|
+--> CLI wrappers and developer tools
+--> Seal manifest and lock material
+--> Panel and installer configuration
+--> docs, examples, receipts, validation transcripts

Seal lane:
  source tree -> manifest/config -> hash baseline -> policy -> report

Installer lane:
```

```
config -> plan -> guarded prefix mutation or dry-run -> receipt -> verification
```

Panel lane:

```
profile -> authority view -> plan/evidence -> guarded action -> status
```

The design becomes easier to reason about when each lane is interpreted by its effects. The Seal lane should mostly read and report. The installer lane may write inside a guarded user-local prefix. The Panel lane may make plans and evidence visible, but it should not silently upgrade itself into root, network, or runtime authority.

4.2 Component seams

The most important design problem is the seam between components. The Seal report may be shown in the Panel. The installer may write a Seal config. CLI wrappers may call Seal. Validation lanes may assert that those calls are safe. Each seam should have an interface contract that states inputs, outputs, authorities, errors, and proof artifacts.

- Seal-to-Panel seam: Panel may invoke report-only checks and display reports; it must not grant enforcement authority.
- Installer-to-Seal seam: installer may install report-only Seal configuration and component markers; it must record receipts.
- CLI-to-Policy seam: every user-facing command must document read effects, write effects, network use, root use, PASS, and FAIL.
- Validation-to-README seam: CI results should constrain public wording and prevent stale status claims.

4.3 Latticra Console as operator surface

Latticra Console, or LC, is the planned operator base for the substrate. Stage 0 makes LC visible without granting host authority: it is panel-installable, configurable, backed by a seed command registry, bound to Runtime Boundary classification plus Seal capability labels, and now extended with inspectable host, receipt, OS-base, and VM-evidence contract surfaces.

- LC command metadata records name, usage, category, effect, capability label, no-effect status, panel visibility, host-process launch status, and future-gate requirements.
- The current report surface is ready-report-only: command registry no-effect is preserved, host process launch is denied, and execution, host mutation, network, runtime enforcement, and boot authority remain zero.
- Future host embedding and OS-base directions are planning profiles only; they do not replace a host OS, launch shell commands, or create boot/runtime authority.
- LC contract commands now include lc host-adapter, lc receipt-request, lc receipt-payload, lc receipt-artifact, lc signature-request, lc os-contract, and lc vm-evidence. They are review surfaces, not effect surfaces.
- The important zeroes are part of the operator contract: host_adapter_present=0, payload_artifact_present=0, seal_signature_request_present=0, receipt_write_allowed=0, vm_evidence_artifact_present=0, boot_authority=0, and production_os_claim=0.

4.4 Nucleus and Nadia as AI-era boundary surfaces

Nucleus and Nadia extend the substrate into AI-era work, but only as boundary surfaces first. Their value is not that they make the project autonomous today; it is that they force task requests, prompt chains, tokenizer artifacts, model readiness, tool authority, and future runtime entry points into explicit metadata before any effect-performing behavior can exist.

- Nucleus currently records classification and policy posture for task-shaped requests. Its report refinements make execution_status=not-executed, effect_status=report-only, runtime_status=not-entered, no_effect_policy=preserved, and representation_gate=language-representation-reviewed visible in deterministic output.
- Nadia is an optional offline-AI foundation, not an active AI assistant. Stage-0 through Stage-27 establish identity, Panel and Console surfaces, local context metadata, runtime/profile contracts, prompt and model contracts,

tokenizer artifact contracts, prompt-tokenization contracts, and prompt-token-sequence contracts, and context-window assembly contracts while keeping model runtime, prompt reading, token creation, context-window assembly, prompt-evaluation-input creation, inference, dialogue generation, network access, source mutation, and tool execution denied.

- The shared rule is evidence before capability. A future AI-adjacent command must pass through command-registry metadata, Nucleus task records, Runtime Boundary labels, Seal report surfaces, and explicit tests before any authority widens.

Chapter 5. Latticra Seal

Seal is the project layer that turns local state into bounded evidence. It is the original center of the handbook and remains the most explicit subsystem.

Seal is not a separate production security product. It is a Latticra substructure for evidence, reporting, policy boundaries, manifests, hash baselines, and future runtime-boundary metadata. Its current intellectual strength is restraint: it begins with describing what exists, measuring what exists, reporting what changed, and refusing to claim authority it does not have.

5.1 The Seal pipeline

Figure 5.1 - Seal pipeline.

```
source tree
-> manifest / seal config
-> hash baseline
-> policy checks
-> report-only evidence
-> Panel / CLI / future runtime-boundary handoff
```

5.2 Manifest layer

The manifest layer defines the legal scope of inspection. A mature manifest should specify included paths, excluded paths, expected files, policy settings, report behavior, and authority declarations. A malformed manifest should not be interpreted generously; it should produce a denial that a developer can fix.

Example 5.1 - Manifest shape for scoped local evidence.

```
# illustrative latticra.seal manifest - not a normative schema
project = "Latticra"
mode = "report-only"

[scope]
include = ["docs", "installer", "examples", "fixtures"]
exclude = [".git", "target", "build", "*.tmp"]

[authority]
network = false
root = false
runtime_enforcement = false
kernel_modification = false
systemd_modification = false
selinux_modification = false

[policy]
fail_closed = true
deny_outside_scope = true
deny_unknown_authority = true
```

5.3 Measurement layer

The measurement layer computes local evidence. In its conservative mode it should read files, compute hashes, compare against the saved baseline, report changed/missing/new files, and avoid mutation unless explicitly running a baseline update command. The measurement layer is not a judge by itself; it produces facts for policy and reports.

Algorithm 5.1 - Read-only measurement skeleton.

```
def measure(manifest, lock):
    assert manifest.mode in {"report-only", "verification"}
    paths = resolve_scope(manifest.scope)
    evidence = []
    for path in paths:
        digest = hash_file(path)
        expected = lock.digest_for(path)
        evidence.append(compare(path, digest, expected))
    return evidence
```

5.4 Policy layer

The policy layer decides whether the measured state is acceptable. The default must be conservative. Unsupported authority claims, malformed manifests, denied paths, and missing metadata should not pass. Policy should be understood as a compiler from messy project state into explicit pass/fail/deny outcomes.

5.5 Report layer

The report layer emits human-readable and eventually machine-readable evidence. The source handbook identifies fields such as timestamp, mode, prefix, kernel, OS, authority flags, manifest/lock presence, hash status, warnings, failures, and final status. A report-only PASS means the checked evidence matched local expectations for that command. It must not be interpreted as proof that the host is secure.

Example 5.2 - Report semantics, with explicit non-claim.

```
timestamp_utc=2026-05-24T00:00:00Z
mode=report-only
prefix=/home/user/.local/share/latticra
network_authority=0
runtime_enforcement_authority=0
root_authority=0
manifest_present=1
lock_present=1
hashes_checked=128
hashes_changed=0
warnings=0
failures=0
status=PASS
meaning=local evidence matched declared expectations; host security not claimed
```

5.6 Seal trust and crypto metadata ladder

Seal now has enough trust-adjacent contract surface to name a ladder without pretending the ladder already grants authority. The stages are signed request metadata, request freshness metadata, verification policy metadata, verification receipt metadata, crypto verify backend metadata, Ed25519 verify-only result metadata, verified receipt promotion metadata, verified capability gate metadata, verified effect decision metadata, runtime handoff evaluation metadata, runtime handoff report metadata, report envelope metadata, and signature request metadata. Each stage records the next question before the project is allowed to answer it with effects.

Signed request records may describe identity, context, parameters, freshness, policy binding, algorithms, key identifiers, and signature hashes, but the signed-request posture is still report-only. Signature generation, signed-request verification, public trust lookup, revocation lookup, signed-request enforcement, host effects, and runtime authority remain zero claims even though a separate local verify-only primitive now exists.

Verification receipts are intentionally weaker than authority. The baseline receipt profile records unverified metadata: `verification_state=unsupported`, `receipt_state=unverified-metadata`, `cryptographic_verification_performed=0`, `verified=0`, `authority_usable=0`, `capability_gate_allowed=0`, and `runtime_authority_granted=0`. A later verified receipt promotion may change the evidence state to `verified`, but not the permission state.

The crypto verify backend remains the metadata boundary that names the provider path. The later Ed25519 verify-only result surface is separate: it can perform a local provider-backed verification over caller-supplied bytes, while the backend contract still carries no trust store, revocation lookup, network behavior, capability authorization, or runtime authority. Post-quantum algorithm labels such as ML-DSA-planned and SLH-DSA-planned remain roadmap vocabulary, not current verification support.

Capability gates, verified effect decisions, runtime handoff reports, report envelopes, and signature requests close the current ladder. A verified capability gate may set `gate_allowed=1`; a verified effect decision may set `effect_allowed=1`; a runtime handoff evaluation may set `handoff_eligible=1`; a runtime handoff report may set `report_ready=1`; a report envelope may set `envelope_ready=1`; and a signature request may set `signature_request_ready=1`. None of those flags means an effect ran, a handoff happened, an object was sealed, a signature was produced, or authority was granted.

5.7 Ed25519 verify-only and verified receipt promotion

The first real cryptographic operation in the current Seal lane is local Ed25519 verify-only. It accepts caller-supplied message bytes, caller-supplied 32-byte Ed25519 public-key bytes, caller-supplied 64-byte signature bytes, and an existing crypto verify backend metadata record. It uses OpenSSL EVP rather than handwritten curve arithmetic, computes a SHA-256 message digest for reporting, and renders deterministic result metadata.

A successful verify-only result may set `crypto_verify_state=verified`, `cryptographic_verification_supported=1`, `cryptographic_verification_performed=1`, `verified=1`, `invalid=0`, and `status=ed25519-verified`. That is a meaningful evidence upgrade, but it is not a trust-store result, not revocation checking, not signing, not private-key handling, not production cryptography certification, and not authority.

Verified receipt promotion consumes only a successful Ed25519 verify-only result and turns it into deterministic verified receipt metadata: `verification_state=verified`, `receipt_state=verified`, `cryptographic_verification_performed=1`, and `verified=1`. The promotion is evidence promotion, not permission promotion. It must preserve `authority_usable=0`, `capability_gate_allowed=0`, `runtime_authority_granted=0`, `capability_authorization_added=0`, `effect_execution_added=0`, host behavior unchanged, and network behavior unchanged.

This distinction is now central to the Seal story. Latticra can say that local verify-only evidence and authority-neutral verified receipt metadata exist. It still must not say that verified receipts authorize capabilities, execute effects, contact trust infrastructure, enforce runtime behavior, or make production security claims.

5.8 Verified capability gate metadata

The verified capability gate consumes verified receipt promotion metadata plus requested capability, effect, and scope labels. Its initial allowlist is intentionally narrow: `verified-receipt-inspection` and `verified-receipt-report` as capability labels, and `report-only` or `evaluate-only` as effect labels.

A successful metadata-only gate may set `gate_allowed=1` and `gate_state=allowed-metadata-only`. That phrase must be read literally. It means the gate evaluation accepted a local metadata request; it does not mean capability enforcement, effect execution, host reads, host writes, network access, tool execution, shell execution, policy persistence, or runtime authority.

The current status shape is therefore subtle and useful: `receipt_state=verified`, `verification_state=verified`, `verified=1`, `gate_allowed=1`, `gate_state=allowed-metadata-only`, `authority_usable=0`, `receipt_capability_gate_allowed=0`, `runtime_authority_granted=0`, `effect_performed=0`, `host_read_performed=0`, `host_write_performed=0`, and `network_performed=0`. The next valid rung is verified effect decision evaluation from an allowed metadata-only gate.

5.9 Verified effect decision metadata

The verified effect decision consumes an allowed verified capability gate and a requested effect label. Its narrow policy accepts only `report-only` and `evaluate-only` requests from `gate_state=allowed-metadata-only`, with `gate_allowed=1` and all `performed/authority` fields still zero.

A successful decision may set `effect_allowed=1` and `decision_state=allowed-report-only` or `decision_state=allowed-evaluate-only`. That is effect classification metadata, not effect execution. It must preserve `effect_performed=0`, `runtime_authority_granted=0`, `host_read_performed=0`, `host_write_performed=0`, and `network_performed=0`.

5.10 Runtime handoff evaluation and report metadata

Runtime handoff evaluation consumes a metadata-only verified effect decision and asks whether it is eligible to be described as a future handoff candidate. A successful evaluation may set `handoff_eligible=1` and `handoff_state=eligible-report-only` or `eligible-evaluate-only` while still preserving `handoff_performed=0` and `runtime_authority_granted=0`.

The runtime handoff report layer consumes eligible handoff evaluation metadata and produces report-readiness metadata. It may set `report_ready=1` and `report_state=ready-report-only` or `ready-evaluate-only`, but it still does not perform a runtime handoff, execute an effect, enforce a capability, read or write host state, contact a network, or grant authority.

5.11 Report envelope and signature request metadata

The report envelope layer consumes ready runtime handoff report metadata and classifies whether a report/evaluate record may be wrapped into deterministic envelope metadata. A successful envelope may set `envelope_ready=1` and `envelope_state=sealed-report-only` or `sealed-evaluate-only`, but the name is deliberately narrow: it is metadata-only, unsigned, object-unsealed, and authority-neutral.

The signature request layer consumes ready report-envelope metadata and classifies an Ed25519-development request label as `signature_request_state=requested-metadata-only` with `signature_request_ready=1`. It is request classification, not signing or verification. It must preserve `signature_performed=0`, `verification_performed=0`, `private_key_handling=0`, `handoff_performed=0`, `effect_performed=0`, `runtime_authority_granted=0`, `host_read_performed=0`, `host_write_performed=0`, and `network_performed=0`.

Chapter 6. Panel, Installer, and Receipts

The Panel and installer are the human-facing path into the substrate. Their job is to make plans, effects, and authority visible before anything effectful happens.

The broader project evidence shows Latticra Panel as a graphical local installer and control panel for Latticra, Lat, LIR, and Latticra Seal. It includes workflows for safe dry-run, guarded user-local installation, verification, and launch. This should be interpreted as a local developer/operator interface, not as a host security authority.

6.1 Why a Panel matters

A command-line tool can document authority, but a graphical installer can either clarify or obscure it. Latticra Panel should be designed to clarify. It should make authority visible in the UI, show whether the current action is dry-run or guarded local-prefix install, surface the plan and receipt paths, and keep network/root/runtime authority disabled unless future work explicitly implements and tests a controlled lane.

Panel action	Expected effect	Boundary
Save configuration	Write local installer config.	Local file write; no host-wide mutation.
Generate plan	Write or refresh install plan.	Evidence output.
Run dry-install	Validate configuration and write dry-run receipt.	No host mutation.
Install guarded local prefix	Write under allowed user-local prefix.	No root/system mutation; receipt required.
Open evidence	Show plan/report/receipt material.	Read/display only.
Change Seal profile	Select intended crypto/report posture.	No signing/encryption unless implemented and tested.

6.2 Guarded local-prefix installation

The installer lane is a concrete example of Latticra thinking. A dry-run can generate a plan and receipt without mutating the host. A guarded local-prefix installation can write under a user-local prefix and install wrappers, docs, examples, component markers, and receipts. It should refuse root authority, network authority, and runtime enforcement authority in the current stage.

Example 6.1 - Current safe installer workflow shape.

```
make -C installer dry-run
make -C installer local-example
make -C installer verify-local
make -C installer uninstall-local

# Expected current posture:
# - user-local prefix
# - no root authority
# - no network authority
# - no kernel/systemd/SELinux modification
# - operator receipt required
```

6.3 Receipts as substrate objects

A receipt is not merely a log. It is a structured witness that an effect happened or would have happened. In Latticra, receipts should say when an action occurred, which mode was used, which prefix was targeted, which plan/config was used, which paths were written, and which authorities were not used.

Receipt field	Reason
timestamp_utc	Makes the action temporally reproducible.
mode	Distinguishes dry-run from local-prefix install.
result	Records success, failure, or denial.
install_prefix	Defines the write boundary.
config and plan	Links decision input to effect output.
installed_paths	Shows exactly which local areas changed.
authority flags	Prevents hidden root/network/runtime assumptions.

Receipts are how Latticra makes installation security a data problem. Instead of asking whether an installer is trusted in the abstract, the reader asks which effects it recorded and whether those effects fit the declared authority vector.

6.4 Platform install lanes

Platform install work belongs in the substrate only when the target, authority, transcript, and rollback story are named. Latticra now has enough macOS and Fedora evidence to describe platform lanes, but the lanes remain deliberately short of production installer readiness.

- The macOS lane is user-local and no-effect today: app bundle writer dry-runs, local candidate probes, commit-gate reports, reset/uninstall dry-run contracts, and verification-transcript contracts define what a future managed writer must prove before any app bundle or Application Support file is written.
- The Fedora lane distinguishes controlled installroot and disposable-VM RPM validation from live-host installation. It can build or inspect local RPM payloads under explicit gates, but Fedora approval, immutable-host support, daily-driver readiness, and production installer readiness stay false.
- The production installer readiness contract is the promotion boundary. It requires reproducible artifacts, checksums, signatures, SBOM/license metadata, install/uninstall/rollback transcripts, unsupported-target blocking, multi-VM validation, and public non-claims before `production_installer_ready` can ever become 1.

6.5 Latticra Console receipt and host contract bridge

LC extends the Panel/install story by installing metadata-only contract files and reports for host embedding, host inventory, host adapter, receipts, receipt requests, payload schema, payload artifact draft, signature-request binding, OS-base planning, and VM evidence. These records make the future promotion path visible before any host adapter, payload artifact, signature request, receipt writer, VM launcher, or OS-base behavior exists.

The bridge is deliberately inert. The installed surfaces can say `request_contract_present=1`, `schema_present=1`, `draft_contract_present=1`, `binding_contract_present=1`, `os_base_contract_present=1`, and `vm_evidence_contract_present=1`, while also preserving `host_adapter_present=0`, `payload_materialized=0`, `signature_request_binding_allowed=0`, `seal_signature_request_present=0`, `receipt_write_allowed=0`, `vm_launch_allowed=0`, and `boot_allowed=0`.

A reviewer should treat these Console records as preconditions for later work: they name the receipts, operator consent, runtime-boundary binding, and Seal capability labels that must exist before host embedding, signing, OS-base, or VM-adjacent behavior can be promoted.

Chapter 7. Lat, LIR, and Contract Surfaces

The Lat and LIR lanes have moved from placeholder component names into a bounded, no-effect contract pipeline: grammar parsing, semantic validation, model normalization, model-aware lowering, and deterministic report/diagnostic surfaces.

A project book should include Lat and LIR without inventing execution semantics. The current disciplined phrasing is: Lat is the contract/declaration lane; LIR is the intermediate representation lane; both are presently evidence and metadata surfaces. They preserve source spans, no-effect flags, declaration and clause metadata, and denial posture so that future runtime work has typed witnesses before it has authority.

7.1 Contract-first explanation

The most useful way to present Lat and LIR today is as names for a language/tooling lane and an intermediate-representation or contract lane. A future edition can replace this conservative explanation with a full specification. For now, the important architectural claim is that they should obey the same Latticra substrate rules as Seal and the installer: declare effects, emit evidence, fail closed on unsupported authority, and keep public claims tied to tests.

7.2 Current no-effect pipeline

As of the May 26 notes, the Lat path is parser -> semantic validation -> normalized model -> model-aware Lat-to-LIR lowering -> pipeline and diagnostic reports -> Runtime Boundary evidence records. This gives the project a stronger internal story than a raw parser: declarations, clauses, transition sources, operators, first-item report metadata, and first-clause provenance can be inspected without evaluating the language.

- Grammar and semantic layers bound the source shape and reject unsupported states before downstream metadata is trusted.
- Model normalization builds typed declaration and clause tables for states, policies, transitions, assertions, and effect declarations.
- Lat-to-LIR lowering consumes the normalized model, emits a `lat_module` LIR shape, and preserves source-span, no-effect, and authority-denial metadata.
- Pipeline and diagnostic reports copy first declaration, first clause, lowering, and error-class metadata for deterministic audit visibility.
- Runtime Boundary now carries the same first-clause provenance at record level: `lat_pipeline_first_clause_node_index`, `role`, `effect`, `name`, `operator`, and `value`. That is evidence propagation, not clause interpretation.
- The first-clause effect label is an observed metadata label from the Lat pipeline, not permission to perform that effect.

The boundary is still explicit: this pipeline does not execute Lat, execute LIR, evaluate operators, mutate state, perform file or network I/O, call recovery behavior, touch hardware, or grant runtime authority.

Surface	Contract questions to answer
lat command	What inputs does it read? What files does it write? What exit codes matter?
LIR artifacts	What is the canonical syntax? What invariants are checked? What is lowered or compiled?
Lat-to-LIR lane	What transformations are allowed? What witnesses are emitted?
Runtime boundary metadata	Which LIR facts can become runtime decisions, and under what future authority?
Tests	Which examples prove stable behavior and which are aspirational?

7.3 Suggested formal shape

Definition 7.1 - Contract skeleton for Lat, LIR, and CLI surfaces.

```
CommandContract = {
  name: String,
  reads: Set<PathOrResource>,
  writes: Set<PathOrResource>,
  authority: AuthorityVector,
  inputs: Schema,
  outputs: Schema,
  pass_condition: Predicate,
  fail_condition: Predicate,
  denial_condition: Predicate,
  receipt_or_report: Optional[Schema]
}
```

The key is not the exact type syntax. The key is the discipline: a Latticra command should not be considered specified until a reader can identify its read set, write set, authority vector, output evidence, and failure modes.

Chapter 8. Field Engine and Research Artifacts

The one-pass field engine is best understood as a research and communication artifact: a visual state lattice with exportable witness material, not a standalone security boundary.

The one-pass field engine material describes a single-file browser demo with deterministic seed behavior, visual state metrics, hashing when browser crypto is available, and an exportable proof manifest. The artifact is useful because it communicates the Latticra aesthetic: state, coherence, entropy, seal, snapshots, and exportable evidence. Its own notes say it is not a standalone security boundary.

8.1 Why include demos in a security substrate?

Security research projects often need artifacts that teach the mental model before the implementation is complete. A visual field engine can make the substrate idea tangible. It shows state evolving, stabilization toggles, snapshots, and exported evidence. But the book must preserve the boundary: a demo can illustrate witness generation without becoming host protection.

Artifact behavior	Substrate lesson
Deterministic seed	Reproducibility matters.
Observable node/edge state	State should be inspectable.
Entropy and coherence metrics	Derived signals can summarize state, but need definitions.
SHA-256 when available	Hashing can bind a witness to observable state.

Exportable manifest	Evidence should be portable.
Explicit non-security note	Demonstrations must not overclaim.

8.2 From demo to research method

The general lesson is that every Latticra artifact should produce a witness. A browser demo exports a manifest. A Seal run emits a report. An installer emits a receipt. A validation lane emits a transcript. A future runtime handoff should emit a decision trace. The forms differ, but the substrate pattern remains stable.

Part III - Security Model

Latticra makes sense when security is modeled as typed effects, explicit authority vectors, and fail-closed policy decisions.

Chapter 9. Authority Vectors and Effect Classes

The authority vector is the core type of the Latticra system substrate. It records which powers a command has and, equally important, which powers it does not have.

An authority vector should be attached to reports, receipts, plans, UI states, and command contracts. It should be boring, explicit, and repeated everywhere. Repetition is a feature: a user should not have to infer whether root, network, runtime, kernel, systemd, or SELinux authority was used.

Definition 9.1 - Current authority vector shape.

```
AuthorityVector = {
    network_authority: Bool,
    runtime_enforcement_authority: Bool,
    root_authority: Bool,
    kernel_modification_performed: Bool,
    systemd_modification_performed: Bool,
    selinux_modification_performed: Bool,
    production_security_product: Bool
}
```

9.1 Deny by type mismatch

If a command is typed as report-only and attempts a network operation, that is an effect mismatch. If an installer profile has `root_authority=false` and attempts to write outside the user-local prefix, that is a boundary violation. If a README claims runtime enforcement but the authority vector says `runtime_enforcement_authority=0`, that is a claim violation. The same model can catch code problems and documentation problems.

Algorithm 9.1 - Authority gate.

```
def authority_gate(requested, allowed):
    for k, v in requested.items():
        if v and not allowed.get(k, False):
            return Deny(reason=f"unsupported authority: {k}")
    return Pass()
```

9.2 Effect taxonomy

The effect taxonomy lets developers reason about commands before implementation. A command that reads a manifest and prints a report has a small effect. A command that writes an installed prefix has a larger effect. A command that modifies systemd or SELinux would need a much stronger trust story and is not part of the current project authority posture.

Type	Allowed examples	Forbidden current examples
ReportOnly	latticra-seal report, dry-run evidence display.	Runtime process blocking.
VerifyLocal	Hash comparison, manifest presence check.	Network reputation lookup.
WriteEvidence	reports, plans, receipts, component markers.	Kernel module installation.
GuardedUserInstall	user-local prefix files and wrappers.	Root package install.
FutureRuntimeHandoff	metadata for a future decision engine.	Claiming sandbox enforcement today.

Chapter 10. Policy as a Fail-Closed Compiler

Policy is the subsystem that turns measured facts and requested effects into decisions. The correct default is fail closed.

A fail-closed policy does not silently pass malformed input, guess intent, grant authority, or claim enforcement. It emits a denial reason. This is exactly the behavior that makes Latticra valuable to a developer: failures are not merely red lights; they are explanations of why the state cannot be trusted or the claim cannot be made.

10.1 Policy input language

A mature policy layer should accept a small, explicit input language: manifest scope, authority vector, requested mode, measured evidence, lock status, command identity, and environment facts. That language should be strict enough to reject unknown authority modes and flexible enough to describe future runtime-boundary metadata without implying enforcement.

Definition 10.1 - Policy input and output sketch.

```
PolicyInput = {
  command: "seal.report",
  mode: "report-only",
  scope: ManifestScope,
  requested_authority: AuthorityVector,
  measured: [MeasuredFile],
  lock_present: Bool,
  manifest_present: Bool
}

PolicyOutput = Pass | Fail(reason) | Deny(reason)
```

10.2 Denial is a feature

In many systems, a denial is treated as a user experience failure. In Latticra, denial is part of the project promise. A policy-denial regression lane proves that unsafe or unsupported states are rejected. That makes denial a positive evidence artifact, not a nuisance.

Condition	Expected policy behavior	Example reason
Malformed manifest	Deny	manifest could not be parsed safely.
Path outside declared scope	Deny	path escapes manifest scope.
network_authority=true	Deny	network authority not implemented.
root_authority=true	Deny	root authority not allowed in current lane.
runtime enforcement claim	Deny	runtime enforcement not implemented or tested.
Changed hash	Fail or report change	current digest differs from lock baseline.
Missing lock	Fail or warn by mode	baseline unavailable for verification.

10.3 Policy and language design

A computer scientist can view Latticra policy as a static analysis over claimed effects. The analyzer has simple goals: reject unknown syntax, reject unsound authority, reject out-of-scope paths, and produce precise errors. This is the right direction because the project is not yet trying to be an all-knowing runtime guard; it is trying to make the static and local story correct.

Chapter 11. Reports, Receipts, and Witness Objects

The substrate is only as useful as the evidence it leaves behind. Latticra should treat every report, receipt, and manifest as a witness object with a schema.

A witness object is a record that lets another program or human reconstruct what was checked, when it was checked, where it was checked, which mode was used, what changed, what failed, and what authority was not used. This is more than logging. A log can be incidental. A witness object is designed.

11.1 Report semantics

A Seal report should be read as a bounded statement about the relationship between current local evidence and declared expectations. PASS means the checked evidence matched the command expectations. FAIL means evidence or policy did not match. DENY means the tool refused to proceed because the request or state was unsafe, unsupported, malformed, or out of scope.

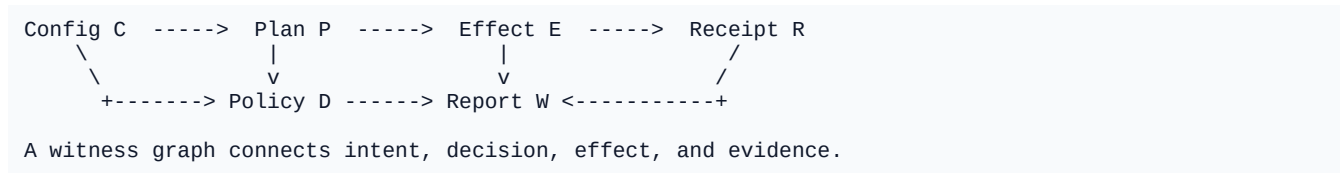
Status	Meaning	Non-meaning
PASS	Checked local evidence matched declared expectations.	Does not mean the host is secure.
FAIL	Evidence did not match or the check could not complete safely.	Does not necessarily imply compromise.
DENY	Policy rejected request or state before accepting it as valid.	Does not mean the project is broken.
WARN	A non-fatal issue should be read by the operator.	Does not grant authority.

11.2 Receipt semantics

Receipts should be constructed so that an auditor can answer two questions: what did the operator ask for, and what did the project actually do? The receipt should include config, plan, target prefix, result, installed paths, and authority flags. In a future signed-evidence model, receipts may also include signatures, key identifiers, or certificate chains, but those should not be claimed until implemented and tested.

11.3 Witness graph

Figure 11.1 - Witness graph.



The witness graph is useful because it generalizes across components. A Seal command has a manifest, a policy decision, a measurement effect, and a report. An installer action has config, plan, guarded writes, and a receipt. A future runtime handoff should have metadata, a capability gate, a decision, and a no-effect dry-run trace before any enforcement claim is allowed.

11.4 Verification receipts and authority handoff

A verification receipt should be treated as a witness object in the graph, not as a permission slip. It records the relationship among artifact digest, signature profile, signer identity label, public-key identity label, verification policy profile, requested receipt mode, and verification state.

The handoff rule is deliberately segmented: signature metadata is not verification; verification is not a receipt; a receipt is not a capability gate; a capability gate is not an effect decision; an effect decision is not runtime handoff; and runtime handoff is not host enforcement. Keeping those nouns separate prevents one implemented layer from borrowing the authority of a future layer.

This also gives reviewers a concrete audit path. If a report claims authority, the witness graph should show a verified receipt, an allowed capability gate, an allowed effect decision, a performed runtime handoff, and explicit evidence for the effect. If any predecessor is missing or unsupported, public wording must fall back to metadata-only, report-only, or denied-by-default language.

11.5 Verified evidence is not permission

The Ed25519 verify-only result changes the witness graph because a report can now contain a real local cryptographic verification result. That result should still be read narrowly: it says the supplied signature matched the supplied message and public key under the local provider boundary.

It does not say the public key is trusted, current, non-revoked, authorized for a capability, or allowed to drive a host effect. Those are different witness edges. A future trust-store edge, revocation edge, capability-gate edge, effect-decision edge, and runtime-handoff edge would each need its own schema, implementation, tests, and public wording.

Verified receipt promotion is therefore useful precisely because it gives the graph a named middle state. A promoted receipt is stronger than unverified metadata, but weaker than authority. Reviewers should treat `authority_usable=0` and `capability_gate_allowed=0` as part of the receipt meaning, not as incidental implementation details.

11.6 Metadata gates before effects

The verified capability gate adds another witness edge between a promoted receipt and an effect decision. Its allowed state is still metadata-only: `gate_allowed=1` is not the same as an effect being allowed, performed, or handed to a runtime.

This gives reviewers a sharper audit rule. A record may show verified receipt evidence, an allowed metadata gate, an allowed metadata-only effect decision, eligible runtime-handoff metadata, report-ready metadata, envelope-ready metadata, and signature-request-ready metadata while still proving that no host read, host write, network access, tool execution, shell execution, capability enforcement, runtime handoff, signing, object sealing, or runtime authority occurred. That is not contradiction; it is the point of the ladder.

11.7 Allowed, eligible, and ready are not performed

The new Seal ladder deliberately uses different verbs for different evidence states. `gate_allowed=1` means a metadata gate accepted the request. `effect_allowed=1` means a metadata-only effect classification accepted the request. `handoff_eligible=1` means a future handoff candidate is describable. `report_ready=1` means a report record can be emitted. `envelope_ready=1` means a deterministic envelope record can be emitted. `signature_request_ready=1` means a future signing path was requested as metadata.

None of those states means `effect_performed=1`, `handoff_performed=1`, `signature_performed=1`, `verification_performed=1`, `private_key_handling=1`, `runtime_authority_granted=1`, object sealing, or host/network behavior. Reviewers should read these flags as named non-effect checkpoints, not as disguised permission grants.

11.8 Envelope and signature request do not sign

Envelope language is risky because ordinary readers may hear "sealed" as cryptographic or object-sealing behavior. In this edition it means only deterministic report-envelope metadata. Signature request language is equally narrow: Ed25519-development is a requested signature label, not a signing operation, private-key action, trust-store decision, or signature verification result.

A reviewer should reject any claim that jumps from `envelope_ready=1` or `signature_request_ready=1` to signing, object sealing, production cryptography, runtime handoff, host mutation, network behavior, or authority. Those transitions require separate contracts, implementations, and denial tests.

11.9 Console receipt requests are not receipts

LC adds a second, component-specific example of the same rule. Its receipt-request, payload-schema, payload-artifact-draft, and signature-request-binding records describe what a future Seal-signed Console receipt would need to contain, but they do not materialize a payload, compute or record a payload hash, create a signature-request artifact, write a receipt, invoke a signer, or verify anything.

The Console witness edge is therefore a request edge, not an evidence edge. The current evidence is the presence of metadata-only contracts and the persistence of denial fields such as `payload_artifact_present=0`, `signature_request_binding_allowed=0`, `seal_signature_request_ready=0`, `receipt_write_allowed=0`, `receipt_signed=0`, `file_write_allowed=0`, and `runtime_authority_granted=0`.

Chapter 12. Threats, Non-Goals, and Honest Boundaries

Honest boundaries are not marketing constraints; they are technical security controls against operator confusion.

A security project that overclaims creates risk. Operators may deploy it where it does not belong. Reviewers may assume protections that do not exist. Developers may build future layers on false premises. Latticra should treat unsupported public claims as a security bug.

12.1 Threat model

The current threat model is defensive and evidence-led. It treats project claims, operator confusion, malformed inputs, unknown requests, unknown effects, failed prerequisites, and future-gated behavior as risks that must be represented before they can be safely relaxed.

- Protected assets include source text integrity, parse results, diagnostic metadata, source spans, semantic validation results, LIR metadata, Lat parse metadata, report records, authority audit records, and project status claims.
- Trust boundaries include parser input, semantic validation, LIR lowering, L-UI rendering, Nucleus task classification, runtime-boundary classification, authority validation, operator reports, and repository documentation.
- External standards are used as source-awareness inputs only; they do not create NSA, CISA, FBI, certification, compliance, or protection claims.
- The validation posture is guarded by scripts and records gaps rather than pretending they are closed by documentation alone.

This turns the threat model into a development constraint: if a boundary is named, the repo should eventually contain a contract, implementation plan, implementation record, guard, status record, and public wording check for that boundary. Until then, the handbook should label the gap clearly.

Threat	How Latticra responds now	Future direction
Project drift	Manifest/lock comparison and report evidence.	Signed baselines and richer schemas.
Operator confusion	Authority flags, receipts, explicit non-claims.	Stronger UI gating and claim linting.
Installer overreach	Guarded user-local prefix and root/network refusal.	Formal installer policy and reproducible packages.
Documentation overclaim	Boundary docs and claim ledger.	CI that checks public wording.
Runtime compromise	Not currently handled as enforcement.	Future runtime-boundary handoff research.
Supply-chain substitution	Hash baselines and receipts begin the story.	Cryptographic signatures and trust roots.

12.2 Non-goals

The current project should not be described as malware prevention, ransomware prevention, a sandbox, a kernel module, an SELinux policy system, a systemd enforcement system, a root installer, a network authority, a runtime enforcement authority, certified host protection, a Fedora-approved package, or an OS replacement. These are not stylistic cautions. They are correctness constraints.

12.3 The boundary rule

Rule 12.1 - Boundary rule.

```
No claim without evidence.
No evidence without test.
No test without documentation.
```

This rule should remain visible in the repository, the Panel, and the handbook. It is the clearest expression of Latticra as a substrate for modern security: the system is allowed to grow, but every growth step must leave a trail.

12.4 Runtime-boundary policy expansion

The runtime-boundary work is the clearest example of how Latticra should mature. After the threat-model refinement named runtime policy as the next evidence gap, the project expanded the policy vocabulary before adding behavior.

- No-effect report, validation, and classification requests remain allowed only under matching report-only, validation-only, or classification-only modes with prerequisites satisfied.
- Operational runtime, host interaction, network interaction, recovery/update/rollback, hardware/boot, unknown, and malformed request families remain denied or future-gated.
- Authority prerequisites must include known request/effect identity, mode compatibility, authority summaries, no-effect flags, deterministic denial reasons, matrix cells, domain cells, and negative tests.
- Lat pipeline prerequisites now include richer provenance: module/count/stage evidence, first-declaration evidence, comment evidence, and record-level first-clause evidence can travel with Runtime Boundary classifications while the request remains denied or no-effect.
- Abuse-case fixtures now cover unknown requests, unknown effects, future-gated execution, operator-confirmation non-override, denial-reason reporting, authority failure, invalid LIR prerequisites, and blocked effects.

The important security property is not that Latticra can execute runtime behavior today. It cannot. The property is that future runtime authority is being forced through named gates, negative tests, report fields, and non-claim review before any promotion is possible.

12.5 AI-era authority and prompt-chain non-goals

AI-facing vocabulary can sound operational long before the system has authority to act. Latticra should therefore treat prompts, tokenizers, model registries, context windows, tool preflights, and task runners as authority-bearing surfaces even when every current implementation remains metadata-only.

- A prompt contract is not prompt evaluation. It may describe source policy, inherited safety requirements, and promotion gates, but it must not imply prompt text receipt, prompt reading, prompt materialization, token creation, runtime invocation, inference, dialogue generation, or tool execution.
- A tokenizer or model contract is not a loaded model. It may describe expected evidence, review fields, and future compatibility, but it must not open tokenizer files, read vocabularies, hash artifacts, attach tokenizers, create runtime sessions, or select model weights unless that behavior is separately implemented and guarded.
- A task record is not task execution. Nucleus may classify, deny, preview, and report, but current records must not mutate state, call a shell, use the network, perform server interaction, recover a host, touch hardware, or enter runtime behavior.

12.6 Host-platform and installer non-goals

Host-platform evidence is especially easy to overread. A dry-run writer, package validation lane, or readiness contract is not the same thing as a supported installer. The handbook should keep those meanings separate.

- A macOS user-local contract may name \$HOME/Applications, Application Support, wrappers, receipts, managed markers, and verification transcript lines, but it must not imply notarization, signing, launchd authority, Keychain access, TCC approval, Endpoint Security, System Extensions, Network Extensions, privileged helpers, or Apple platform approval.
- A Fedora RPM lifecycle lane may mutate a temporary installroot or disposable VM under explicit gates, but it must not imply live developer-host mutation, sudo readiness, Fedora distribution acceptance, immutable Fedora support, service activation, kernel modules, SELinux policy, boot changes, or daily-driver eligibility.
- Installer language must preserve the evidence level: dry-run, contract, installroot, disposable VM, manual release-candidate checklist, and production installer readiness are different claims and should never collapse into one public sentence.

Part IV - Implementation Workbook

The book now turns from architecture to practical engineering patterns that can guide future Latticra work.

Chapter 13. Command Contracts

Every command in Latticra should have a contract that a reviewer can understand without reading all of the source code.

The Seal handbook already states the right documentation posture: every command should say what it reads, what it writes, what it refuses to do, whether it is report-only, whether it requires root, whether it uses the network, what counts as PASS, and what counts as FAIL. This chapter turns that into a reusable command contract template.

Template 13.1 - Latticra command contract.

```
# Command contract template
name:
summary:
mode:
reads:
writes:
refuses:
authority:
  root: false
  network: false
  runtime_enforcement: false
  kernel_modification: false
  systemd_modification: false
  selinux_modification: false
exit_codes:
  0: pass
  1: fail
  2: policy_denial_or_bad_input
pass_meaning:
fail_meaning:
denial_meaning:
reports_or_receipts:
examples:
```

13.1 Example: seal report

Field	Value
name	latticra-seal report
mode	report-only
reads	local manifest, lock material, selected local files, OS facts
writes	report output only, if configured
refuses	network authority, root authority, runtime enforcement authority
PASS	checked local evidence matched declared expectations
FAIL	missing/malformed evidence, mismatch, or safe completion failure

13.2 Example: installer dry-run

Field	Value
name	make -C installer dry-run
mode	dry-install metadata
reads	installer config, component manifest, local repo files
writes	plan and receipt artifacts
refuses	host mutation, root authority, network authority
PASS	plan/receipt generated and authority constraints preserved

FAIL	invalid config, missing scripts, unsafe prefix, or tooling error
------	--

13.3 Command review checklist

- Can the read set be named without vague phrases such as "system state"?
- Can the write set be inspected and removed?
- Can a policy denial be reproduced with a minimal test fixture?
- Does the command use the network, directly or indirectly?
- Would a reasonable user infer a stronger security claim than the command proves?
- Does the README wording match the command contract?

13.4 Latticra Console registry contract

LC turns the command-contract idea into an explicit registry surface. A command is not just a string in a help menu; it is a metadata record that can be reviewed by Panel, Runtime Boundary, Seal, docs, and future tooling before any host-facing behavior exists.

- Allowed Stage-0 commands are help, status, plan, save, dry-run, reset, uninstall, clear, LC inspection commands, and navigation placeholders, all under a metadata/report boundary.
- The registry-backed help and manpage renderers expose command names, categories, effects, and capability labels from the same seed metadata used by installed wrappers.
- Future-facing commands such as `lc host` and `lc os` are intentionally future-gated: they can describe planned authority boundaries, but they cannot launch host processes or claim OS-base behavior.
- Contract commands such as `lc host-adapter`, `lc receipt-request`, `lc receipt-payload`, `lc receipt-artifact`, `lc signature-request`, `lc os-contract`, and `lc vm-evidence` expose promotion gates and denial fields as command output.
- Their source and installed report surfaces must remain paired: `latticra_console_report <surface>` and `latticra-lc <surface>` should tell the same no-effect story for host, receipt, signature, OS, and VM planning metadata.
- The review rule is simple: adding a command means adding its effect label, capability label, no-effect flag, future-gate status, report surface, and denial behavior before widening authority.

13.5 Nucleus and Nadia command surfaces

The Nucleus and Nadia command surfaces should be read as report producers and contract generators. Their command names are intentionally useful to operators, but the commands remain bounded by the metadata they emit and by the authority they explicitly do not have.

- Nucleus commands must expose request kind, effect label, policy result, denial reason, execution status, effect status, runtime status, no-effect policy, and representation gate. A successful report is not proof that a task ran; it is proof that the boundary was classified.
- Nadia commands under the embedded Console and installed `latticra-nadia` wrapper must keep the current stage visible. Status, context-pack, runtime-profile, prompt-plan, tool-preflight, prompt, model, tokenizer, token-sequence, and context-window commands are metadata or contract surfaces until a later contract proves otherwise.
- Command review should reject any help text, status output, README prose, or Panel label that implies autonomous coding, model inference, prompt evaluation, tokenizer attachment, runtime sessions, network access, source mutation, or tool execution ahead of evidence.

Chapter 14. Schemas and Formats

Latticra should prefer small, stable schemas over prose-only state. Schemas are how future tools can consume today's evidence.

The first schema family should cover manifests, reports, receipts, policy-denial reasons, and runtime-handoff metadata. The schemas should be simple enough for shell and Rust tooling, but strict enough to reject ambiguity. TOML, JSON, and line-oriented key/value formats can all be useful, provided the parser behavior is documented.

14.1 Manifest schema requirements

- Schema version and project identifier.
- Mode: report-only, verification, or future explicitly gated modes.
- Scope: include/exclude path lists with normalized semantics.
- Authority vector: explicit booleans for all sensitive authorities.
- Policy requirements: fail-closed behavior and denial classes.
- Report behavior: output path, format, and human-readable summary behavior.

14.2 Report schema requirements

- Timestamp, host facts, mode, prefix, command identity.
- Manifest and lock presence plus validation status.
- Hash counts: checked, changed, missing, new, ignored.
- Authority flags with zero values preserved, not omitted.
- Warnings and failures as structured arrays with stable codes.
- Final status and explicit PASS/FAIL/DENY meaning.

14.3 Denial code taxonomy

Code	Meaning
DENY_UNKNOWN_AUTHORITY	An unsupported authority key or mode was requested.
DENY_NETWORK_AUTHORITY	Network authority was requested in a lane that has none.
DENY_ROOT_AUTHORITY	Root authority was requested in a lane that has none.
DENY_RUNTIME_ENFORCEMENT	Runtime enforcement was requested or claimed without implementation.
DENY_SCOPE_ESCAPE	A path resolves outside declared scope or guarded prefix.
DENY_MALFORMED_MANIFEST	Manifest could not be parsed or lacks required fields.
DENY_UNSUPPORTED_CRYPTOCCLAIM	A signing/encryption claim is not backed by implemented verification.

14.4 Runtime-handoff metadata

Future runtime-boundary handoff metadata should be designed before enforcement exists. The metadata can describe decisions in no-effect dry-run form: what would be denied, why, under which capability gate, with which evidence. The handoff schema must not imply that a runtime enforcement path exists until it does.

Definition 14.1 - Future no-effect runtime handoff sketch.

```
RuntimeHandoffDryRun = {
  schema_version: 1,
  mode: "no-effect-dry-run",
  capability_gate: "future-runtime-boundary",
```

```

input_report: "path-or-digest",
decision: "would-deny" | "would-allow" | "inconclusive",
reason_codes: [String],
enforcement_performed: false
}

```

14.5 Prompt and tokenizer metadata ladder

Nadia gives the handbook a concrete example of schema-first AI design. Each stage records the next boundary without crossing it, so the ladder is useful even before inference exists.

- Foundation and local-context records bind component identity, local install paths, context-pack generation, runtime profile metadata, workbench plans, mode labels, and productivity-ledger vocabulary to local, operator-invoked evidence.
- Prompt and model records bind prompt-evaluation contracts, local model registry contracts, inference-readiness contracts, runtime-invocation contracts, model-load contracts, prompt receipts, prompt materialization, and prompt-evaluation handoff to denied runtime authority.
- Tokenizer records bind tokenizer specification, manifest, artifact inventory, measurement, verification, binding, and runtime-attachment contracts without opening tokenizer files, loading vocabularies, attaching tokenizers, or creating runtime sessions.
- Prompt-tokenization and prompt-token-sequence records reserve the future shape of token count, token order, offsets, masks, and position IDs; Stage-27 context-window records then reserve future context-window layout, token budget, truncation, and prompt-evaluation-input requirements while explicitly recording that those values are not produced today.

14.6 Seal trust metadata schemas

The trust ladder should use small, stable key/value schemas before it grows richer object formats. The current schema vocabulary includes `signed_request_profile`, `freshness_profile`, `receipt_profile`, `verify_profile`, `ed25519_verify_profile`, `verification_policy_profile`, `backend_profile`, `gate_profile`, `decision_profile`, `signature_profile`, `manifest_profile`, `artifact_digest_algorithm`, `message_digest_algorithm`, `signer_identity_label`, `public_key_identity_label`, `trust_source`, `requested_capability`, `requested_effect`, `requested_scope`, and `status`.

State fields should remain explicit even when the value is a denial. The important fields include `signed_request_supported`, `signature_present`, `signature_valid`, `request_freshness_supported`, `replay_protection_supported`, `verification_state`, `receipt_state`, `crypto_verify_state`, `verified`, `invalid`, `authority_usable`, `capability_gate_allowed`, `gate_state`, `gate_allowed`, `decision_state`, `effect_allowed`, `effect_performed`, and `runtime_authority_granted`.

Effect fields must not disappear when false. Report consumers should see `host_read_performed=0`, `host_write_performed=0`, `network_performed=0`, `cryptographic_verification_performed=0`, `verification_performed=0`, `signature_performed=0`, `signer_invoked=0`, `trust_store_loaded=0`, `revocation_lookup_performed=0`, `key_material_loaded=0`, `private_key_handling=0`, `capability_enforcement_performed=0`, and `runtime_handoff_performed=0` where the corresponding layer is in scope.

The schema rule is simple: a future field may reserve vocabulary, but a current field must not imply capability. If the project adds trust stores, revocation checks, signed envelopes, capability authorization, effect execution, or production cryptography claims later, each transition should change only after a contract, implementation, invariant tests, status record, public wording update, and handbook update are in place.

14.7 Verify-only result and promoted receipt schemas

The Ed25519 verify-only result schema is the first place where cryptographic verification fields can legitimately become one. Its key fields are `ed25519_verify_profile`, `backend_profile`, `verification_policy_profile`, `message_label`, `message_size_bytes`, `message_digest_algorithm`, `message_digest_hex`, `public_key_identity_label`, `public_key_size_bytes`, `signature_algorithm`, `signature_size_bytes`, `trust_source`, `crypto_verify_state`, `cryptographic_verification_supported`, `cryptographic_verification_performed`, `verified`, `invalid`, `authority_usable`, `capability_gate_allowed`, `runtime_authority_granted`, and `status`.

The promoted receipt schema copies the evidence-bearing fields into receipt form and changes the receipt state, not the authority state. Its important fields include `receipt_profile`, `verify_profile`, `backend_profile`, `verification_policy_profile`, `message_label`, `message_digest_algorithm`, `message_digest_hex`, `public_key_identity_label`, `signature_algorithm`, `trust_source`, `verification_state=verified`, `receipt_state=verified`, `verified=1`, `authority_usable=0`, `capability_gate_allowed=0`, `runtime_authority_granted=0`, `mode=verified-receipt-authority-neutral`, and `status=verified-receipt-metadata`.

Schema consumers should distinguish mathematical verification from trust. The current `trust_source=local-test-vector` label is evidence about the fixture boundary, not a trust root, revocation service, certificate chain, or production identity system.

14.8 Verified capability gate schema

The verified capability gate schema binds a verified receipt to a requested metadata capability without crossing into enforcement. Core fields include `gate_profile`, `receipt_profile`, `verify_profile`, `message_digest_algorithm`, `message_digest_hex`, `public_key_identity_label`, `receipt_state`, `verification_state`, `requested_capability`, `requested_effect`, `requested_scope`, `verified`, `authority_usable`, `receipt_capability_gate_allowed`, `gate_allowed`, `gate_state`, `runtime_authority_granted`, `effect_performed`, `host_read_performed`, `host_write_performed`, `network_performed`, and `status`.

The important distinction is that `gate_allowed=1` and `gate_state=allowed-metadata-only` may coexist with `authority_usable=0`, `receipt_capability_gate_allowed=0`, `runtime_authority_granted=0`, and `effect_performed=0`. Schema consumers must not collapse metadata permission into runtime permission.

14.9 Verified effect, handoff evaluation, and handoff report schemas

The verified effect decision schema carries `decision_profile`, `gate_profile`, `receipt_profile`, `requested_capability`, `requested_effect`, `requested_scope`, `gate_allowed`, `gate_state`, `decision_state`, `effect_allowed`, `effect_performed`, `runtime_authority_granted`, `host_read_performed`, `host_write_performed`, `network_performed`, `error`, and `status`. The central invariant is that `effect_allowed=1` may coexist with `effect_performed=0` and `runtime_authority_granted=0`.

The runtime handoff evaluation schema adds `handoff_profile`, `requested_handoff`, `handoff_state`, and `handoff_eligible` while preserving `handoff_performed=0`. The runtime handoff report schema adds `report_profile`, `requested_report`, `report_state`, and `report_ready`. Schema consumers must not collapse `handoff_eligible=1` or `report_ready=1` into runtime handoff execution.

14.10 Report envelope and signature request schemas

The report envelope schema adds `envelope_profile`, `requested_envelope`, `envelope_state`, `envelope_ready`, and `signature_performed` to the runtime handoff report lineage. Its central invariant is that `envelope_ready=1` may coexist with `signature_performed=0`, `handoff_performed=0`, `effect_performed=0`, `runtime_authority_granted=0`, and no host or network behavior.

The signature request schema adds `signature_request_profile`, `requested_signature`, `signature_request_state`, `signature_request_ready`, `verification_performed`, and `private_key_handling`. The current allowed request label is `Ed25519-development`, but that label is metadata-only. Schema consumers must not interpret `signature_request_ready=1` as `Ed25519` signing, key generation, private-key handling, trust establishment, or signature verification.

14.11 Latticra Console contract schemas

LC contract schemas should stay small and auditable. The host-adapter contract centers fields such as `contract_profile=lc-host-adapter-v0`, `host_adapter_enabled`, `host_adapter_present`, `host_embedding_contract_receipt_required`, `host_inventory_contract_receipt_required`, `operator_consent_required`, `runtime_boundary_required`, `seal_capability_labels_required`, and `promotion_gate`.

The receipt path centers `request_profile=lc-receipt-request-v0`, `schema_profile=lc-receipt-payload-schema-v0`, `draft_profile=lc-receipt-payload-artifact-draft-v0`, `binding_profile=lc-signature-request-binding-v0`, `requested_receipt_profile`, `requested_capability`, `requested_surfaces`, `payload_fields`, `required_authority_fields`, `required_prior_contracts`, and `signing_authorization_profile`. Schema consumers must preserve `payload_artifact_present=0`, `payload_materialized=0`, `payload_hash_computed=0`, `seal_signature_request_present=0`, `receipt_write_allowed=0`, and `receipt_signed=0` until later artifacts exist.

The OS and VM planning contracts center `contract_profile=lc-os-base-v0`, `contract_profile=lc-vm-evidence-v0`, `vm_evidence_required`, `vm_evidence_capture_enabled`, `vm_evidence_artifact_present`, `os_base_enabled`, `production_os_claim`, `read_only_host_inventory_receipt_required`, `receipt_required_before_os_base`, `receipt_required_before_vm_evidence`, and `promotion_gate`. These fields reserve evidence obligations; they do not grant boot, hypervisor, disk-image, guest, host, network, or runtime authority.

14.12 Lat clause evidence in Runtime Boundary records

Runtime Boundary Lat pipeline records should keep provenance fields separate from authority fields. The current first-clause record shape includes `lat_pipeline_first_clause_node_index`, `lat_pipeline_first_clause_role`, `lat_pipeline_first_clause_effect`, `lat_pipeline_first_clause_name`, `lat_pipeline_first_clause_operator`, and `lat_pipeline_first_clause_value`.

The schema rule is that role, effect, operator, and value are copied evidence. They do not mean the operator was evaluated, the clause was enforced, the effect was performed, LIR executed, file/network I/O occurred, or `runtime_authority_granted` changed from zero.

Chapter 15. Test Lanes and Continuous Evidence

Tests are not just quality gates in Latticra. They are the mechanism that keeps claims from exceeding evidence.

The current project already includes or references useful lanes: Seal smoke checks, policy-denial checks, installer dry-run, guarded local install verification, uninstall verification, Rust formatting/checks for Panel installer code, shell script validation, Fedora VM CLI/RPM validation, and CI status tracking. The next step is to turn these into a continuous evidence architecture.

Lane	Purpose	Evidence expected
make seal	Smoke-check local Seal behavior.	Report or command result.
make seal-policy-denials	Prove unsafe policies are denied.	Denial transcript and status.
installer dry-run	Validate plan without host mutation.	Plan and dry-run receipt.
verify-local	Check guarded user-local install.	Installed paths and Seal report generation.
uninstall-local	Remove managed files.	Removal transcript and absence checks.
Panel Rust check	Validate GUI installer crate builds.	cargo fmt/check output.
Fedora VM/RPM lane	Research local packaging and validation.	VM transcript with non-claims preserved.
make latticra-console	Validate LC registry, Panel installability, host/receipt/signature/OS/VM contract reports, and boundary metadata.	Console report transcript with required contract-present fields and no-effect denial fields.
Lat pipeline and Runtime Boundary evidence	Validate Lat first-clause provenance through pipeline reports and Runtime Boundary records without changing authority.	Lat pipeline and Runtime Boundary transcripts with stable clause metadata and no-effect denial fields.

15.1 Evidence-level testing

A normal test says "the code behaved." An evidence-level test says "the code behaved, emitted the expected witness, and did not claim an unsupported authority." For Latticra, that second form is the more important one.

Example 15.1 - Evidence-level assertions.

```
assert report.status == "PASS"
assert report.network_authority == 0
```

```
assert report.root_authority == 0
assert report.runtime_enforcement_authority == 0
assert "host is secure" not in report.public_summary.lower()
```

15.2 Claim linting

A future CI lane should lint README files, docs, Panel text, and release notes for forbidden phrases such as production-ready, ransomware-proof, malware-proof, kernel-enforced, runtime-enforced, certified, or guaranteed secure. The linter should fail unless the claim ledger says the corresponding evidence and tests exist.

15.3 Disposable validation environments

Host-level validation should remain isolated and explicit. Disposable Fedora VM lanes are useful because they separate validation from daily-driver risk. They also allow the project to say exactly what was validated without claiming Fedora approval, production readiness, or immutable-host compatibility.

15.4 Nucleus and Nadia validation lanes

The AI-era validation lane should prove non-effects as aggressively as it proves positive metadata. The important evidence is often the zero: no runtime entered, no prompt read, no model loaded, no tokenizer opened, no task executed, and no tool authority granted.

- Nucleus focused tests should keep report-only execution labels, no-effect alignment labels, future-gated behavior, denied mutations, and disabled server/recovery/hardware/runtime behavior stable across refactors.
- Nadia focused tests should cover Stage-0 through Stage-27 contract outputs, inherited protective-safety fields, installed command names, Panel/Console metadata, and denial fields for model, tokenizer, prompt, runtime, network, source-mutation, and tool authority.
- A failing validation lane should weaken public wording before it widens capability. In this substrate, conservative status is a feature, not a lack of ambition.

15.5 Platform validation lanes

Platform validation should be treated as evidence stratification. The question is not merely whether a command succeeded; it is where the effect occurred, what was allowed to mutate, and which larger readiness claim remained denied.

- macOS tests should prove path guards, user-local target selection, managed-marker requirements, closed commit gates, verification-transcript contract fields, reset/uninstall dry-run ordering, and zero host mutation while no writer exists.
- Fedora tests should distinguish static spec checks, installroot lifecycle evidence, disposable-VM local RPM validation, manual host release-candidate classification, and production-readiness blockers.
- A platform validation report should always carry negative fields: root_authority=0, network_performed=0, production_installer_ready=0, platform_approval_claimed=0, and daily_driver_install_ready=0 unless a later evidence package proves otherwise.

15.6 Seal trust-boundary validation lanes

Seal trust-boundary tests should prove the negative path as carefully as the happy path. The signed-request lane should still assert that signed-request support, generation, signed-request verification, signature-present, signature-valid, trust-store, revocation, network, host-effect, and runtime-authority fields remain zero until those specific layers exist.

The request-freshness lane should cover timestamp and nonce vocabulary without claiming trusted clocks, nonce storage, replay caches, freshness validation, replay protection, context hashing, parameter hashing, or enforcement. The report surface is valuable precisely because it makes the absence of those mechanisms reproducible.

The baseline verification-receipt and crypto-backend lanes should keep unverified and unsupported predecessor states stable: verification_state=unsupported, receipt_state=unverified-metadata, crypto_verify_state=unsupported,

cryptographic_verification_performed=0, verified=0, authority_usable=0, and runtime_authority_granted=0. The Ed25519 and promotion lanes then prove the later, narrower evidence upgrade separately.

The capability-gate and effect-decision lanes should assert denied defaults: gate_state=denied-unverified, gate_allowed=0, decision_state=denied-gate, effect_allowed=0, effect_performed=0, host_read_performed=0, host_write_performed=0, network_performed=0, and runtime_authority_granted=0. When those tests fail, the default response should be to weaken or fix the claim, not to widen authority.

15.7 Ed25519 and promotion validation lanes

The Ed25519 verify-only lane should prove both the real positive and the safe negative cases. The positive case verifies the RFC8032-style test vector and renders crypto_verify_state=verified, cryptographic_verification_performed=1, verified=1, authority_usable=0, capability_gate_allowed=0, and runtime_authority_granted=0.

The negative cases are just as important: tampered signature, unsupported algorithm, missing message, oversized message, missing public key, wrong public-key size, missing signature, wrong signature size, provider failure, null output, null report buffers, and tiny report buffers must fail closed without key generation, signing, trust-store access, revocation lookup, host reads, host writes, network access, capability enforcement, or runtime authority.

The verified receipt promotion lane should accept only a successful Ed25519 verify-only result. It should reject unsupported verification, verification-not-performed, verified=0, invalid=1, wrong crypto state, missing digest, and missing public-key identity. Its success case must keep authority_usable=0, capability_gate_allowed=0, and runtime_authority_granted=0 visible in both the record and report.

15.8 Verified capability gate validation lane

The verified capability gate lane should prove that the narrow allowlist works and that the boundaries remain visible. Valid report/evaluate requests may set gate_allowed=1 and gate_state=allowed-metadata-only while preserving runtime_authority_granted=0, effect_performed=0, host_read_performed=0, host_write_performed=0, and network_performed=0.

The failure lane should reject null receipts, invalid receipts, unverified receipts, invalid receipt flags, missing digest metadata, missing public-key identity, missing requested capability, unknown capability, missing requested effect, unknown effect, null output, null reports, and tiny report buffers. None of those failures may execute effects, persist policy, contact networks, read or write host files, execute tools, execute shells, enforce capabilities, or grant runtime authority.

15.9 Effect decision and handoff validation lanes

The verified effect decision lane should accept only allowed metadata gates and report/evaluate effects. It should prove effect_allowed=1 only when the gate is valid, gate_state=allowed-metadata-only, runtime authority is zero, and no host, network, handoff, or effect has already occurred.

The runtime handoff evaluation and report lanes should accept only metadata-only allowed decisions, then prove handoff_eligible=1 and report_ready=1 without performing handoff or granting authority. Negative cases should cover null inputs, invalid predecessors, unsupported labels, mismatched report requests, prior performed effects, host read/write flags, network flags, tiny report buffers, and runtime_authority_granted=1.

15.10 Envelope and signature request validation lanes

The report envelope lane should accept only ready runtime handoff reports and report/evaluate envelope labels, then prove envelope_ready=1 without signing, object sealing, performing handoff, performing effects, reading host state, writing host state, contacting networks, or granting runtime authority.

The signature request lane should accept only ready sealed-report or sealed-evaluate envelope metadata and the Ed25519-development request label, then prove signature_request_ready=1 while signature_performed=0, verification_performed=0, private_key_handling=0, handoff_performed=0, effect_performed=0, runtime_authority_granted=0, and host/network flags remain zero. Negative cases should cover invalid envelopes, missing or unknown signature labels, prior signature/effect/handoff flags, tiny report buffers, and runtime_authority_granted=1.

15.11 Latticra Console contract validation lane

The Console validation lane should prove both halves of the contract bundle: the positive presence of reportable metadata and the negative absence of authority. The current evidence target is make latticra-console plus focused checks for host-adapter, receipt-request, payload schema, payload artifact draft, signature-request binding, OS-base, VM-evidence, boundary, help, manpage, installed wrapper, and Panel component metadata.

The no-effect assertions are the important part: `command_registry_no_effect=1`, `host_process_launch_allowed=0`, `host_adapter_present=0`, `payload_artifact_present=0`, `seal_signature_request_present=0`, `receipt_write_allowed=0`, `vm_evidence_artifact_present=0`, `vm_launch_allowed=0`, `boot_allowed=0`, `production_os_claim=0`, `network_authority=0`, and `runtime_enforcement_authority=0`.

15.12 Lat pipeline and Runtime Boundary evidence validation

The Lat and Runtime Boundary validation lanes should prove that provenance survives without changing authority. Current evidence targets include `sh scripts/test-lat-pipeline-report-refinement.sh`, `sh scripts/test-lat-pipeline.sh`, and `sh scripts/test-runtime-boundary.sh`.

The important checks are field stability and non-effect preservation: first-clause role/effect/name/operator/value metadata may be copied, while operator evaluation, Lat execution, LIR execution, file I/O, network I/O, host mutation, hardware behavior, boot behavior, and runtime authority stay denied.

Chapter 16. Roadmap and Research Agenda

The roadmap should grow in disciplined layers. Each phase adds a stronger substrate property without skipping evidence.

The original Seal roadmap is a good foundation: documentation baseline, CLI clarity, manifest and lock hardening, report schema, Panel integration, cryptographic strengthening, runtime-boundary handoff, and Fedora/Linux integration research. The whole-project roadmap should use the same layering but apply it across Latticra components.

Phase	Goal	Done when
1. Documentation baseline	Make the project understandable.	Every component has role, status, non-claims, and command map.
2. CLI clarity	Make commands predictable.	Read/write/refuse/root/network/PASS/FAIL contracts exist.
3. Manifest and lock hardening	Make local verification reproducible.	Schema, lock format, update rules, ignored paths, changed output, malformed tests.
4. Report schema	Make evidence human- and machine-readable.	Required fields, warning/failure classes, stable status values.
5. Panel integration	Make Seal and installer evidence visible.	Panel exposes report-only scan, plans, receipts, manifest/lock state, no authority escalation.
6. Cryptographic strengthening	Move from hashes to signed evidence.	Signing model, key handling, trust roots, invalid signature tests.
7. Runtime-boundary handoff	Prepare metadata without overclaiming.	No-effect dry-run handoff, verified capability gate, verified effect decision, handoff/report/envelope/signature-request readiness, and denial tests.
8. Fedora/Linux integration research	Explore host integration carefully.	User-local evidence, no-root assumptions, RPM boundaries, SELinux/systemd non-claims.
9. LC host/receipt/OS promotion gates	Promote Console planning surfaces only after receipted evidence exists.	Host adapter, receipt payload, signature request, OS-base, and VM-evidence artifacts each have schemas, tests, receipts, and non-claims before authority widens.

10. Lat provenance to runtime-boundary records	Keep language evidence inspectable before any runtime promotion.	First-clause, first-declaration, comment, stage, module, and count evidence remain stable across Lat pipeline and Runtime Boundary tests with no execution authority.
--	--	---

16.1 Research questions

- What is the smallest manifest schema that can support useful reproducible evidence?
- Can public claim linting be tied directly to the authority vector and test results?
- What receipt format is expressive enough for local installs but simple enough for shell tooling?
- How should a future trust root be introduced without weakening the current no-claim posture?
- What runtime-boundary metadata can be generated today that remains honest about no enforcement?
- Can Panel UX make authority boundaries intuitive without hiding the formal model?

16.2 The long-term shape

The long-term Latticra system should be a substrate where every artifact is typed by its effects, every security-relevant action leaves a witness, every public claim is checked against evidence, and every future authority upgrade passes through a capability gate. That is a meaningful contribution to modern security even before it becomes a production enforcement system.

Appendices

Reference material for developers, reviewers, and project writers.

Appendix A. Glossary

A compact vocabulary for the Latticra substrate.

Term	Meaning
Authority vector	A structured record of which powers a command or lane has or explicitly lacks.
Boundary	A limit on scope, authority, or claim strength.
Claim ledger	The project record of what is implemented, evidenced, planned, or forbidden to claim.
Effect	An observable action such as reading, writing, installing, networking, or enforcing.
Evidence-bound	Constrained by reproducible evidence and tests.
Fail closed	Reject unsafe, malformed, unsupported, or contradictory input instead of guessing.
Manifest	A declaration of inspection scope and policy expectations.
Receipt	A witness for an operator action or installer effect.
Report-only	A mode that inspects and describes local state without runtime enforcement.
Runtime handoff	Future metadata path for enforcement decisions, not current enforcement.
Seal	The Latticra evidence, reporting, and policy-boundary layer.
Substrate	A foundational layer of state, policy, evidence, and authority relations.
Verification receipt	A witness record for verification metadata; not authority unless later cryptographic verification and gate policy are implemented and tested.
Capability gate	A boundary record that may later decide whether a verified receipt can request a capability; currently denied for unverified metadata.
Effect decision	A record that separates an allowed gate from any actual host, network, runtime, or tool effect.
Verify-only result	A local cryptographic verification result for caller-supplied bytes that remains separate from signing, trust, capability, and effects.
Evidence promotion	A transition that strengthens a witness record, such as from a verified result to a verified receipt, without granting authority.
Metadata gate	A policy evaluation record that can allow metadata-only inspection or reporting without performing an effect.
Verified effect decision	A metadata-only classification that may set <code>effect_allowed=1</code> for report/evaluate requests while keeping <code>effect_performed=0</code> .
Handoff eligibility	A metadata-only runtime-boundary classification that may set <code>handoff_eligible=1</code> without performing a handoff.
Report readiness	A metadata-only report classification that may set <code>report_ready=1</code> without runtime authority or host/network behavior.
Report envelope	A deterministic metadata envelope around ready report metadata; currently unsigned and object-unsealed.
Signature request	A metadata-only request for a future signing path that does not sign, verify, handle keys, or grant authority.
Host-adapter contract	A metadata-only LC record naming the future host embedding adapter preconditions while preserving <code>host_adapter_present=0</code> and denying host authority.
Receipt request contract	A metadata-only LC record that asks for a future Seal receipt path without materializing a payload, writing a receipt, signing, or verifying.
OS-base planning contract	A metadata-only LC record naming future OS-base evidence obligations while denying boot, kernel, hardware, host, network, and runtime authority.
VM evidence contract	A metadata-only LC record naming future VM evidence obligations while denying VM launch, hypervisor

	access, disk-image mutation, guest networking, host mutation, and boot authority.
Clause evidence	Copied Lat clause metadata such as role, effect label, name, operator, value, and node index; useful for audit but not operator evaluation or authority.
Provenance field	A schema field that records where evidence came from or how it was classified without changing the allowed effect state.

Appendix B. Reviewer Checklists

Use these checklists when reviewing pull requests, release notes, or public project pages.

B.1 Documentation checklist

- Does the text distinguish evidence from enforcement?
- Does it preserve current non-claims around root, network, runtime, kernel, systemd, SELinux, and production readiness?
- Does every command document read/write/refuse behavior?
- Are PASS and FAIL meanings bounded and precise?
- Are future features labeled as future work rather than current behavior?

B.2 Implementation checklist

- Does the code normalize paths before comparing scope?
- Does malformed policy fail closed?
- Does a report or receipt include authority flags even when all are zero?
- Does a dry-run avoid host mutation?
- Does a local install stay inside the guarded prefix?
- Are denial reasons stable enough for regression tests?

B.3 Panel UX checklist

- Does the UI show whether the current action is dry-run or guarded install?
- Does it make authority visible before an action?
- Does it show plan and receipt paths?
- Does it avoid language that implies production protection?
- Does it distinguish crypto posture selection from actual signing/encryption behavior?

Appendix C. Public Wording Guide

Project language should be rigorous enough that a reader can infer the implementation boundary.

Use this kind of wording	Avoid this kind of wording
early verification and reporting layer	production security platform
local evidence and policy-boundary subsystem	malware or ransomware prevention
report-only state inspection	runtime enforcement
manifest/hash baseline mechanism	kernel-enforced protection

guarded user-local installer	root installer
future runtime-boundary research	sandbox today
policy-regression-backed denial behavior	guaranteed secure

A strong public sentence: "Latticra is an early, local, evidence-bound substrate for report-only verification, policy regression, manifest/hash baselines, authority-neutral Ed25519 verification evidence, metadata-only Seal gate/effect/handoff/report/envelope/signature-request classification, metadata-only Console host/receipt/OS/VM contract surfaces, Lat/LIR provenance records, guarded user-local installation, and future runtime-boundary research."

A sentence to reject: "Latticra protects production systems from malware with kernel-enforced runtime isolation." That sentence exceeds current evidence.

Appendix D. Source Basis for This Edition

This section records how this draft was grounded.

This edition was derived from the Latticra Seal handbook and recent repository material covering signed-request, freshness, verification, Seal metadata ladder extensions through signature request, Console host-adapter/receipt-request/payload/signature/OS/VM contract surfaces, Runtime Boundary Lat pipeline first-clause evidence propagation, installer, Panel, Nucleus, Nadia, macOS, Fedora/RPM, Lat/LIR, and one-pass field-engine evidence. It distinguishes implemented facts from inferred substrate architecture and preserves project non-claims.

Source area	Used for
Seal handbook	Seal role, architecture, policy, reports, boundaries, and roadmap.
Panel installer material	Project-level component map, UI/installer workflow, authority visibility.
Installer receipts and dry-run evidence	Guarded user-local installation and receipt semantics.
Fedora/RPM validation snippets	Validation lane framing with explicit non-claims.
One-pass field engine	Research artifact and witness-manifest pattern.
Latticra Console foundation	Stage-0 command registry, Panel installability, runtime-boundary binding, host-adapter contract, receipt-request/payload/signature-request-binding contracts, OS-base planning, VM-evidence contract, and no-effect operator-surface posture.
Nucleus task boundary	Report-only task classification, no-effect alignment, execution-status labels, and future-gated task vocabulary.
Nadia offline AI contracts	Stage-0 through Stage-27 metadata ladder for Panel/Console visibility, local context, model/prompt/tokenizer contracts, prompt-token-sequence and context-window boundaries, and denied AI/tool authority.
macOS user-local app bundle lane	Dry-run writer posture, user-local app bundle contract, commit gate, reset/uninstall contract, and verification transcript requirements.
Fedora RPM validation lanes	Installroot and disposable-VM RPM lifecycle evidence, bounded host-facing validation vocabulary, and Fedora non-claims.
Production installer readiness contract	Promotion gates for future installer artifacts, signatures, SBOM, install/uninstall/rollback transcripts, multi-VM validation, and readiness non-claims.
Seal trust and crypto metadata ladder	Signed request, freshness, receipt/backend, Ed25519 verify-only, verified receipt, gate, effect, handoff, report, envelope, and signature-request metadata boundaries.
Seal authority non-claims	Readiness flags remain separated from signing, object sealing, effect execution, host/network behavior, and runtime authority.
Runtime Boundary Lat clause evidence	Record-level propagation of first lowered clause node, role, effect, name, operator, and value while preserving no-effect runtime posture.

Future editions should replace inferred connective tissue with canonical specs for root architecture, Lat/LIR, Panel, Console, Nucleus, Nadia, Seal, installers, platform validation, reports, receipts, and runtime handoff.

Quick Start: How to Read Latticra as a Computer Scientist

1. Start with the authority vector. Identify what the tool is allowed to do and what it explicitly cannot do.
2. Read the manifest and scope rules. A local evidence tool is only meaningful if its observation set is defined.
3. Inspect the report or receipt. Look for timestamp, mode, prefix, authority flags, warnings, failures, and status meaning.
4. Review denial tests. A good policy layer proves that unsafe states do not pass.
5. Compare documentation to evidence. Delete or weaken any sentence that outruns implementation and tests.
6. Treat future runtime-boundary work as metadata research until an enforcement path exists and is tested.
7. For AI-era surfaces, distinguish contract metadata from model loading, prompt tokenization, inference, dialogue generation, source mutation, and tool execution.
8. For platform work, distinguish dry-run, installroot, disposable-VM, manual-host, and production-installer evidence.
9. For Seal work, follow the ladder through verified receipt, gate, effect, handoff, report, envelope, and signature-request metadata; never skip from evidence to authority.
10. For Console work, distinguish contract-present metadata from adapter presence, payload materialization, signature requests, receipt writing, VM launch, OS-base behavior, and boot authority.
11. For Lat/LIR work, distinguish provenance fields from language execution: copied clause operators and effect labels are evidence, not evaluated predicates or permissions.